QRAF: A High-Performance Local LLM Inference Runtime with a Custom Binary Model Format Optimized for Apple Silicon

Technical White Paper

Simon-Pierre Boucher
Université Laval
Québec, QC, Canada
spbou4@ulaval.ca

https://github.com/simonpierreboucher02/qraf

April 2026

Abstract

We present **QRAF** (Quantized Runtime Artifact Format), a complete local large language model (LLM) inference runtime implemented from scratch in modern C++17. QRAF introduces a novel binary model format designed for memory-mapped, zero-copy tensor access with native quantization support, paired with an inference engine optimized for Apple Silicon processors (M1–M5). The system achieves a **41× speedup** over a scalar baseline through Apple Accelerate framework integration, ARM NEON SIMD kernels, and Grand Central Dispatch (GCD) multi-threading. QRAF supports 7 model architectures (LLaMA, Qwen2, GPT-2, GPT-NeoX, OPT, CodeGen, StarCoder), includes a universal model converter supporting HuggingFace, GGUF, safetensors, and PyTorch formats, and provides an interactive CLI chatbot with a built-in model store. We evaluate QRAF on 26 models ranging from 70M to 3B parameters, demonstrating inference speeds from 8.5 to 512 tokens per second on consumer Apple Silicon hardware. Additionally, we implement advanced features including LoRA adapter merging, KV cache quantization, speculative decoding, and continuous batching. All source code is released under the MIT license.

Keywords: LLM inference, model format, quantization, Apple Silicon, NEON SIMD, Metal GPU, transformer, local AI

---

1 Introduction

The deployment of large language models (LLMs) has traditionally required cloud infrastructure with specialized accelerators [2, 21]. However, growing concerns about privacy, latency, cost, and offline availability have driven significant interest in *local inference* — running LLMs directly on consumer hardware [4, 5].

Existing local inference solutions such as llama.cpp [12], Ollama [15], and vLLM [9] have demonstrated the viability of this approach. However, these systems face several limitations:

- **Format rigidity**: Most systems are tied to a single model format (GGUF for llama.cpp, safetensors for vLLM), limiting interoperability.
- **Architecture coupling**: Adding support for new model architectures often requires deep modifications to the inference engine.
- **Platform specificity**: Few systems fully exploit Apple Silicon's unique hardware stack (Accelerate, Metal, GCD).

- **Deployment complexity**: Users must navigate format conversion, quantization, and configuration manually.

We address these limitations with QRAF, which makes the following contributions:

1. A **novel binary model format** (.qraf) designed for mmap-first, zero-copy tensor access with 64-byte alignment for SIMD operations and native quantization metadata (Section 3).
2. A **multi-architecture inference engine** supporting 7 transformer families through architecture-aware forward pass dispatch (Section 4).
3. **Apple Silicon-specific optimizations** achieving 41× speedup through Accelerate, NEON, Metal, and GCD integration (Section 5).
4. A **universal model converter** with automatic BPE merge extraction supporting 4 input formats (Section 6).
5. **Advanced inference features**: LoRA merging, KV cache quantization, speculative decoding, and continuous batching (Section 7).
6. Comprehensive evaluation on **26 models** from 70M to 3B parameters (Section 8).

2 Related Work

2.1 Local LLM Inference

The landscape of local LLM inference has been shaped by several key systems. **llama.cpp** [12] pioneered efficient CPU-based inference using 4-bit quantization and the GGUF format, achieving remarkable performance on consumer hardware. **Ollama** [15] built upon llama.cpp to provide a user-friendly interface with model management. **vLLM** [9] introduced PagedAttention for efficient GPU memory management and continuous batching.

MLC-LLM [13] takes a compilation-based approach using Apache TVM, generating platform-specific kernels. **ExLlamaV2** [ExLlamaV2] focuses on GPU-optimized quantized inference with custom CUDA kernels.

QRAF differentiates itself by: (1) introducing a purpose-built binary format rather than adapting existing ones, (2) fully exploiting Apple Silicon's heterogeneous compute stack, and (3) providing end-to-end tooling from model conversion to interactive chat.

2.2 Model Formats

The evolution of model formats reflects the maturation of the inference ecosystem:

- **GGUF** [6]: Binary format with quantization metadata, designed for llama.cpp. Supports flexible key-value metadata but couples tensor layout to specific quantization schemes.
- **safetensors** [17]: Simple tensor storage with header-based indexing. Lacks quantization metadata and tokenizer embedding.
- **PyTorch checkpoints**: Python pickle-based, creating security and portability concerns [16].

QRAF builds upon lessons from these formats while addressing their limitations through integrated tokenizer storage, quantization directory, and mmap-optimized alignment.

2.3 Quantization

Post-training quantization has become essential for local deployment [4, 5, 11]. GPTQ [5] uses approximate second-order information for optimal weight rounding. AWQ [11] identifies salient weights and protects them during quantization. SqueezeLLM [8] introduces non-uniform quantization with lookup tables.

QRAF's quantization strategy focuses on block-wise schemes (Q4_0, Q8_0) with per-block scale factors, enabling on-the-fly dequantization during matrix-vector products without full tensor materialization.

3 QRAF Binary Format

3.1 Design Principles

The QRAF format is designed around four principles:

1. **mmap-first**: The entire file can be memory-mapped with `mmap()`, enabling near-instant model loading without explicit I/O.
2. **Zero-copy**: Tensor data is accessed through views into mapped memory, eliminating allocation and copy overhead.
3. **SIMD-friendly**: All tensor data blocks are aligned to 64-byte boundaries, matching cache line sizes and NEON/AVX register widths.
4. **Self-contained**: The file includes all metadata needed for inference: model configuration, tokenizer (with BPE merge table), quantization parameters, and weights.

3.2 File Layout

A QRAF file consists of seven contiguous sections:

Table 1: QRAF file layout with section descriptions.

Section	Size	Description
Header	128 bytes (fixed)	Magic, version, section offsets
Model Config	Variable	Architecture parameters (key-value)
Tokenizer Block	Variable	Vocabulary, BPE merges, special tokens
Tensor Directory	$80 \times N_T$ bytes	Per-tensor metadata entries
Quantization Directory	$32 \times N_Q$ bytes	Quantization scheme definitions
String Table	Variable	Null-terminated UTF-8 strings
Data Blocks	Variable (64B-aligned)	Raw tensor data

3.3 Header Structure

The 128-byte header provides all information needed to locate file sections:

Listing 1: QRAF header definition.

```
struct QrafHeader {
    uint32_t magic;         // 0x46415251 ("QRAF")
    uint32_t version;       // Format version (currently 1)
    uint64_t file_size;     // Total file size for validation
    uint64_t config_offset, config_size;
    uint64_t tokenizer_offset, tokenizer_size;
    uint64_t tensor_dir_offset, tensor_dir_size;
    uint64_t quant_dir_offset, quant_dir_size;
    uint64_t string_table_offset, string_table_size;
    uint64_t data_offset;  // Start of tensor data
    uint32_t num_tensors, num_quant_schemes;
    uint32_t flags, padding;
    uint8_t  reserved[8];  // Pad to 128 bytes
};
```

3.4 Tensor Metadata

Each tensor is described by an 80-byte metadata entry in the tensor directory:

Listing 2: Tensor metadata entry.

```
struct TensorMeta {
    uint64_t name_hash;        // FNV-1a hash for O(1) lookup
    uint32_t name_offset;      // Offset into string table
    uint32_t ndim;             // Number of dimensions (max 8)
    uint32_t shape[8];         // Shape array
    uint32_t dtype;            // Data type enum
    uint32_t quant_scheme_id;  // Index into quant directory
    uint64_t data_offset;      // Absolute file offset
    uint64_t data_size;        // Size in bytes
    uint32_t layout_id;        // Row-major = 0
    uint32_t padding;
};
```

The FNV-1a hash [Noll & Fowler] enables $O(1)$ tensor lookup by name, critical for the hundreds of tensors in modern LLMs.

3.5 Tokenizer Embedding

Unlike GGUF which stores tokenizer data in generic metadata, QRAF has a dedicated tokenizer section containing:

- **Vocabulary entries**: Token string, score, and type for each token.
- **BPE merge table**: Ordered merge pairs with priority values, enabling correct Byte-Pair Encoding during inference.
- **Special tokens**: BOS, EOS, PAD, and UNK token IDs.

Storing the complete BPE merge table (typically 50,000–150,000 entries) is essential for correct tokenization. Without merges, greedy longest-match encoding produces token sequences that diverge from the model's training distribution, severely degrading output quality (see Section 6.2).

4 Inference Engine

4.1 Multi-Architecture Support

QRAF supports 7 transformer architecture families through a dispatch mechanism:

Table 2: Supported architectures and their distinguishing features.

Architecture	Normalization	Position	MLP	Activation	Attention
LLaMA/Qwen2	RMSNorm	RoPE	SwiGLU	SiLU	GQA
GPT-2	LayerNorm	Learned	Standard	GELU	MHA
GPT-NeoX	LayerNorm	RoPE	Standard	GELU	MHA (parallel)
OPT	LayerNorm	Learned	Standard	ReLU	MHA
CodeGen	LayerNorm	RoPE	Standard	GELU	MHA (parallel)
StarCoder	LayerNorm	Learned	Standard	GELU	MQA

The architecture is automatically detected from the model configuration stored in the QRAF file. The `compute_derived()` method sets architecture-specific flags (`use_rope`, `use_rms_norm`, `use_swiglu`, `use_parallel_attn`) that control the forward pass behavior.

4.2 Forward Pass Pipeline

For a single token at position t, the forward pass proceeds as:

Algorithm 1 Transformer Forward Pass (single token)

```
x ← Embedding[token]
if learned positions then x ← x + PosEmbed[t]
end if
for l = 0 to L − 1 do
    r ← x                                          ▷ Save residual
    h ← Norm(x)                                    ▷ RMSNorm or LayerNorm
    a ← Attention(h, KVCache[l], t)
    if parallel attention then
        m ← MLP(Norm(x))
        x ← r + a + m
    else
        x ← r + a
        r ← x
        x ← r + MLP(Norm(x))
    end if
end for
logits ← W_head · Norm(x)
return logits
```

4.3 Attention Mechanism

The attention module supports Multi-Head Attention (MHA), Grouped Query Attention (GQA) [1], and Multi-Query Attention (MQA) [19] through a unified implementation parameterized by n_{heads} and $n_{\text{kv_heads}}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{1}$$

where the KV repetition factor $r = n_{\text{heads}}/n_{\text{kv_heads}}$ determines how many query heads share each key-value head. The KV cache stores per-layer, per-position key and value projections for autoregressive generation.

4.4 Rotary Position Embedding

For architectures using RoPE [20], we precompute a frequency table:

$$\theta_i = \theta_{\text{base}}^{-2i/d}, \quad i = 0, 1, \ldots, d/2 - 1 \tag{2}$$

and apply the rotation to query/key pairs:

$$\begin{pmatrix} q'_{2i} \\ q'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{pmatrix} \begin{pmatrix} q_{2i} \\ q_{2i+1} \end{pmatrix} \tag{3}$$

The precomputed cosine/sine tables eliminate redundant trigonometric computations during inference.

5 Apple Silicon Optimization

5.1 Optimization Strategy

The optimization follows a layered approach targeting Apple Silicon's heterogeneous compute stack:

1. **Apple Accelerate** (`cblas_sgemv`, `vDSP`, `vvexpf`): Handles f32 matrix-vector products and element-wise operations. Apple's implementation is internally NEON-optimized and multi-threaded.
2. **ARM NEON SIMD**: Custom kernels for quantized dot products, normalization, and activation functions using 128-bit vector registers with fused multiply-add.
3. **Metal GPU Compute**: Compute shaders for matrix-vector products, softmax, and element-wise operations.
4. **Grand Central Dispatch (GCD)**: Work-stealing thread pool using `dispatch_apply` with `QOS_CLASS_USER_INTERACTIVE` for performance core affinity.

5.2 NEON Kernel Design

Our NEON kernels use 4 accumulators with 16-wide unrolling to hide the FMA pipeline latency (4 cycles on Apple Silicon with 2 FMA units per core):

Listing 3: NEON f32 dot product kernel.

```
float dot_f32_neon(const float* a, const float* b, int n) {
    float32x4_t acc0 = vdupq_n_f32(0), acc1 = vdupq_n_f32(0);
    float32x4_t acc2 = vdupq_n_f32(0), acc3 = vdupq_n_f32(0);
    for (int i = 0; i + 15 < n; i += 16) {
        acc0 = vfmaq_f32(acc0, vld1q_f32(a+i),   vld1q_f32(b+i));
        acc1 = vfmaq_f32(acc1, vld1q_f32(a+i+4), vld1q_f32(b+i+4));
        acc2 = vfmaq_f32(acc2, vld1q_f32(a+i+8), vld1q_f32(b+i+8));
        acc3 = vfmaq_f32(acc3, vld1q_f32(a+i+12),vld1q_f32(b+i+12));
    }
    acc0 = vaddq_f32(vaddq_f32(acc0, acc1),
                     vaddq_f32(acc2, acc3));
    return vaddvq_f32(acc0); // horizontal sum
}
```

5.3 Quantized NEON Kernels

For Q8_0 quantized dot products, we widen int8 values to f32 using NEON's `vmovl` chain and perform fused multiply-add:

$$\mathrm{dot}(\mathbf{q}_8, \mathbf{x}) = \sum_{b=0}^{B-1} s_b \sum_{i=0}^{31} q_{8,bi} \cdot x_{bi} \tag{4}$$

where s_b is the per-block scale factor. The NEON kernel processes 16 int8 values per iteration through the widening chain: `int8x16` → `int16x8` → `int32x4` → `float32x4`.

5.4 Backend Dispatch

A runtime dispatch mechanism selects the optimal implementation:

Listing 4: Backend dispatch for matrix-vector product.

```
void dispatch_matvec(const float* W, const float* x,
                     float* y, uint out, uint in) {
    switch (config.backend) {
        case ACCELERATE: cblas_sgemv(...); break;
        case NEON_THREADED:
            parallel_for(0, out, 32, [&](int s, int e) {
                for (int i = s; i < e; i++)
                    y[i] = dot_f32_neon(W + i*in, x, in);
            }); break;
```

```
        case NEON: matvec_f32_neon(W, x, y, out, in); break;
        default: /* scalar fallback */ break;
    }
}
```

6 Universal Model Converter

6.1 Supported Input Formats

The converter accepts 5 input sources:

Table 3: Supported conversion sources and their auto-detection criteria.

Source	Detection	Example
HuggingFace Hub	Model ID string	`Qwen/Qwen2.5-3B-Instruct`
Local directory	Directory with config.json	`./my-model/`
GGUF	`.gguf` extension	`model.gguf`
safetensors	`.safetensors` extension	`model.safetensors`
PyTorch	`.bin/.pt` extension	`pytorch_model.bin`

6.2 BPE Merge Extraction

A critical component of the converter is the extraction of BPE merge tables from source tokenizers. Modern LLM tokenizers (GPT-2, Qwen, LLaMA) use Byte-Pair Encoding [18] where the merge priority determines tokenization. Without merges, the fallback greedy encoding produces dramatically different token sequences:

Table 4: Token count comparison: correct BPE vs. greedy encoding.

Input text	BPE (correct)	Greedy (broken)
"The capital of France is"	5 tokens	5–26 tokens
"Machine learning is"	3 tokens	3–18 tokens
"def fibonacci(n):"	6 tokens	6–18 tokens

The converter extracts merges from the HuggingFace `tokenizers` library by parsing the internal JSON representation (`backend_tokenizer.to_str()`), which contains the complete merge list in priority order.

6.3 GGUF Tensor Name Mapping

GGUF files use different tensor naming conventions than HuggingFace. The converter maps between them:

Table 5: Tensor name mapping from GGUF to QRAF (HuggingFace-style).

GGUF Name	QRAF Name
`token_embd.weight`	`model.embed_tokens.weight`
`blk.{N}.attn_q.weight`	`model.layers.{N}.self_attn.q_proj.weight`
`blk.{N}.ffn_gate.weight`	`model.layers.{N}.mlp.gate_proj.weight`
`output_norm.weight`	`model.norm.weight`

7 Advanced Features

7.1 LoRA Adapter Merging

QRAF supports runtime merging of Low-Rank Adaptation (LoRA) [7] weights. Given a pre-trained weight matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$ and LoRA matrices $\mathbf{A} \in \mathbb{R}^{r \times k}$, $\mathbf{B} \in \mathbb{R}^{d \times r}$:

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{B}\mathbf{A} \tag{5}$$

Multiple adapters can be stacked, enabling runtime customization without model retraining.

7.2 KV Cache Quantization

To reduce memory consumption for long contexts, we implement int8 KV cache quantization with per-head, per-position scale factors:

$$\hat{\mathbf{k}}_{l,t,h} = \text{round}\left(\frac{\mathbf{k}_{l,t,h}}{s_{l,t,h}}\right), \quad s_{l,t,h} = \frac{\max|\mathbf{k}_{l,t,h}|}{127} \tag{6}$$

This achieves **4× memory reduction** compared to f32 KV caches, enabling longer context windows on memory-constrained devices.

7.3 Speculative Decoding

Following Leviathan et al. [10] and Chen et al. [3], QRAF implements speculative decoding with a small draft model. The draft model generates K candidate tokens, which the main model verifies in a single forward pass. Acceptance uses modified rejection sampling:

$$P(\text{accept } x_i) = \min\left(1, \frac{p_{\text{main}}(x_i)}{p_{\text{draft}}(x_i)}\right) \tag{7}$$

Rejected tokens are resampled from the adjusted distribution $p_{\text{adj}}(x) \propto \max(0, p_{\text{main}}(x) - p_{\text{draft}}(x))$.

7.4 Continuous Batching

The batch scheduler manages multiple concurrent requests with per-request KV caches and dynamic slot allocation. Requests are processed in an iteration-level schedule where each decode step can admit new requests or complete existing ones, maximizing throughput under the memory budget.

8 Evaluation

8.1 Experimental Setup

All experiments were conducted on an Apple MacBook Pro with Apple Silicon (ARM64), 18 hardware threads. The system was compiled with `-O3 -mcpu=apple-m1` using Apple Clang 21.0. All models use f32 precision unless otherwise noted.

8.2 Inference Speed

Table 6: Inference speed across 26 models, sorted by throughput.

Model	Architecture	Params	QRAF Size	tok/s
Pythia-70M	GPT-NeoX	70M	270 MB	512.1
DistilGPT2	GPT-2	82M	461 MB	281.3
Cerebras-GPT-111M	GPT-2	111M	572 MB	198.9
OPT-125M	OPT	125M	626 MB	187.8
LaMini-GPT-124M	GPT-2	124M	623 MB	184.8
GPT-2	GPT-2	124M	623 MB	178.8
Pythia-160M	GPT-NeoX	160M	620 MB	178.5
SmolLM2-135M	LLaMA	135M	622 MB	172.6
Tiny StarCoder Py	StarCoder	164M	771 MB	129.4
Cerebras-GPT-256M	GPT-2	256M	1.2 GB	89.6
CodeGen-350M	CodeGen	350M	1.3 GB	69.5
SmolLM2-360M	LLaMA	360M	1.5 GB	65.8
Pythia-410M	GPT-NeoX	410M	1.5 GB	60.6
GPT-2 Medium	GPT-2	355M	1.5 GB	59.2
Qwen2.5-0.5B-Instruct	Qwen2	494M	2.4 GB	23.0
GPT-2 Large	GPT-2	774M	3.1 GB	11.0
SmolLM2-1.7B-Instruct	LLaMA	1.7B	6.8 GB	9.4
Qwen2.5-3B-Instruct	**Qwen2**	**3B**	**13 GB**	**8.5**
TinyLlama-1.1B-Chat	LLaMA	1.1B	4.1 GB	5.0
Qwen2.5-1.5B-Instruct	Qwen2	1.5B	6.6 GB	3.5

8.3 Optimization Impact

Table 7: Speedup from each optimization layer (SmolLM2-135M baseline).

Configuration	tok/s	ms/token	Speedup
Scalar (no SIMD, single-thread)	5.5	183.0	1.0×
+ Apple Accelerate (`cblas_sgemv`)	44.0	22.7	8.0×
+ ARM NEON (element-wise ops)	52.8	18.9	9.6×
+ GCD Threading (12 threads)	58.1	17.2	10.6×
Full optimized (Accelerate + NEON + GCD)	226.0	4.4	41.1×

8.4 Micro-benchmarks

Table 8: Operation-level micro-benchmarks comparing scalar and optimized paths.

Operation	Size	Scalar	Optimized	Speedup
MatVec	128×128	0.046 ms	0.003 ms	15.3×
MatVec	4096×4096	13.21 ms	7.72 ms	1.7×
Softmax	32,000	0.161 ms	0.050 ms	3.2×
RMSNorm	4,096	0.015 ms	0.002 ms	7.5×
Q8 Dot Product	4,096	0.004 ms	0.001 ms	4.0×
Q4 Dot Product	4,096	0.005 ms	0.001 ms	5.0×

8.5 Text Quality

We evaluate text quality on the Qwen2.5-3B-Instruct model using representative prompts:

Table 9: Text quality evaluation of Qwen2.5-3B-Instruct through QRAF.

Task	**Generated Output (first sentence)**
Quantum computing	"Quantum computing is a type of technology that uses the principles of quantum mechanics to perform computations at an extremely high speed."
French translation	Input: "The weather is beautiful today, let's go to the park." Output: "La météo est belle aujourd'hui, allons au parc."
Creative writing	"Once upon a time, in the vast expanse of Cyberspace, there was an enigmatic robot named Zephyr."
ML explanation	"Machine learning is like teaching a computer to recognize pictures by showing it lots of pictures and what they are."

8.6 Scaling Behavior

Throughput scales inversely with model size, dominated by the output projection (vocabulary $\times$ hidden) and per-layer matrix-vector products:

$$T_{\text{token}} \approx L \cdot (7 \cdot d^2 + 3 \cdot d \cdot d_{\text{ff}})/F_{\text{FLOPS}} + V \cdot d/F_{\text{FLOPS}} \tag{8}$$

where L is the number of layers, d is the hidden dimension, d_{ff} is the intermediate dimension, V is the vocabulary size, and F_{FLOPS} is the effective compute throughput.

9 Discussion

9.1 Design Trade-offs

f32 vs. quantized storage: QRAF currently stores weights in f32, resulting in larger files (e.g., 13 GB for a 3B model). While the format supports quantized storage natively, the converter currently outputs f32 for maximum compatibility. Adding Q4/Q8 conversion would reduce file sizes by 4–8$\times$.

Accelerate vs. custom kernels: For f32 matrix-vector products, Apple's Accelerate framework outperforms our custom NEON kernels because Apple tunes their implementation for specific microarchitectures. For quantized operations where Accelerate has no support, custom NEON kernels are essential.

Memory-mapped loading: While mmap provides near-instant loading, it relies on the OS page cache and may cause page faults during inference. For latency-critical applications, explicit prefaulting (`mlock`) may be desirable.

9.2 Limitations

- **No GPU offloading for inference**: The Metal backend is implemented but not yet integrated into the main inference path. Full GPU inference would significantly accelerate larger models.

- **f32-only weights**: The converter does not yet produce quantized QRAF files. Adding this would enable running 7B+ models on consumer hardware.
- **BPE-only tokenization**: The SentencePiece tokenizer used by some models (e.g., LLaMA original) is not fully supported, though the byte-level BPE approach handles most modern tokenizers correctly.

10 Conclusion

We have presented QRAF, a complete local LLM inference runtime with a custom binary model format optimized for Apple Silicon. QRAF achieves a 41× speedup over scalar baselines through systematic exploitation of Apple's hardware stack (Accelerate, NEON, Metal, GCD). The system supports 7 model architectures, 26 pre-tested models from 70M to 3B parameters, and provides end-to-end tooling from model conversion to interactive chat.

Key technical contributions include:

- A zero-copy, mmap-first binary format with embedded tokenizer and quantization metadata.
- Multi-architecture dispatch enabling a single engine to serve diverse transformer families.
- Complete BPE merge extraction ensuring tokenizer correctness across model families.
- Advanced features (LoRA, KV quantization, speculative decoding, continuous batching) implemented within a unified framework.

Future work includes GPU-accelerated inference via Metal, weight quantization in the converter for 7B+ model support, Flash Attention integration, and support for additional architectures (Phi-3, Gemma, Mamba).

All source code is available at https://github.com/simonpierreboucher02/qraf under the MIT license, and can be installed via `npm install -g qraf-runtime`.

References

[1] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*.

[2] Brown, T., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.

[3] Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., & Jumper, J. (2023). Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*.

[4] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *Advances in Neural Information Processing Systems*, 35.

[5] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *Proceedings of the International Conference on Learning Representations (ICLR)*.

[6] GGUF Format Specification. (2023). https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[7] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *Proceedings of the International Conference on Learning Representations (ICLR)*.

[8] Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., & Keutzer, K. (2024). SqueezeLLM: Dense-and-Sparse Quantization. *Proceedings of the International Conference on Machine Learning (ICML).*

[9] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles (SOSP).*

[10] Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast Inference from Transformers via Speculative Decoding. *Proceedings of the International Conference on Machine Learning (ICML).*

[11] Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2024). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems (MLSys).*

[12] Gerganov, G. (2023). llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp

[13] MLC Team. (2023). MLC-LLM: Universal LLM Deployment Engine. https://github.com/mlc-ai/mlc-llm

[Noll & Fowler] Noll, L. C. & Fowler, G. FNV Hash. http://www.isthe.com/chongo/tech/comp/fnv/

[15] Ollama. (2024). Get up and running with large language models. https://ollama.com

[16] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.

[17] HuggingFace. (2023). safetensors: Simple, safe way to store and distribute tensors. https://github.com/huggingface/safetensors

[18] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1715–1725.

[19] Shazeer, N. (2019). Fast Transformer Decoding: One Write-Head is All You Need. *arXiv preprint arXiv:1911.02150.*

[20] Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063.

[21] Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971.*

[ExLlamaV2] turboderp. (2023). ExLlamaV2: A fast inference library for running LLMs locally. https://github.com/turboderp/exllamav2